RDK Multi-Tenant E-Commerce Platform - Complete Rebuild Plan

Version: 3.0 (Final, Locked)
Date: February 9, 2026
Author: System Architect
Status: Ready for Implementation

Executive Summary

Project Overview

What: Rebuild RDK from single-tenant Next.js/Supabase to multi-tenant Go/PostgreSQL platform supporting unlimited branded sneaker reseller storefronts.

Why: Multiple resellers want similar sites. Instead of building separate codebases, create one platform where each tenant gets their own custom domain with unique branding.

Architecture: Single backend codebase, single frontend codebase, multi-domain routing, complete tenant isolation.

Target Scale: 500K MAU across 100+ tenants, <500ms response time, 99.9% uptime

Table of Contents

1. Technology Stack
2. Architecture Overview
3. Complete Cost Analysis
4. Database Schema
5. API Design
6. Frontend Architecture
7. Backend Folder Structure

8. Frontend Folder Structure
9. Implementation Timeline
10. Development Workflow
11. Deployment Strategy
12. Security & Performance
13. Migration Plan
14. Success Metrics

Technology Stack

Backend

Component	Technology	Version	Why
Language	Go	1.22+	2.5x faster than Node, better concurrency, lower memory
Framework	Chi	v5.1.0	Lightweight, fast routing, middleware support
Database	PostgreSQL	16	Multi-tenancy support, partitioning, JSON columns
ORM/Query	sqlc	Latest	Compile-time safe SQL, no reflection overhead
Migrations	Goose	Latest	Simple, reliable SQL migrations
Auth	Clerk	Latest	Multi-tenant orgs, saves 2-3 weeks dev time
Payments	Stripe Connect	Latest	Direct charges to tenant accounts
Search	Meilisearch	v1.11+	10-50x faster than Postgres FTS
Cache	Upstash Redis	Latest	Serverless Redis, global edge
Storage	Cloudflare R2	Latest	S3-compatible, zero egress fees
Email	AWS SES	Latest	$0.10 per 1K emails
Jobs	Asynq	Latest	Redis-backed background jobs

Frontend

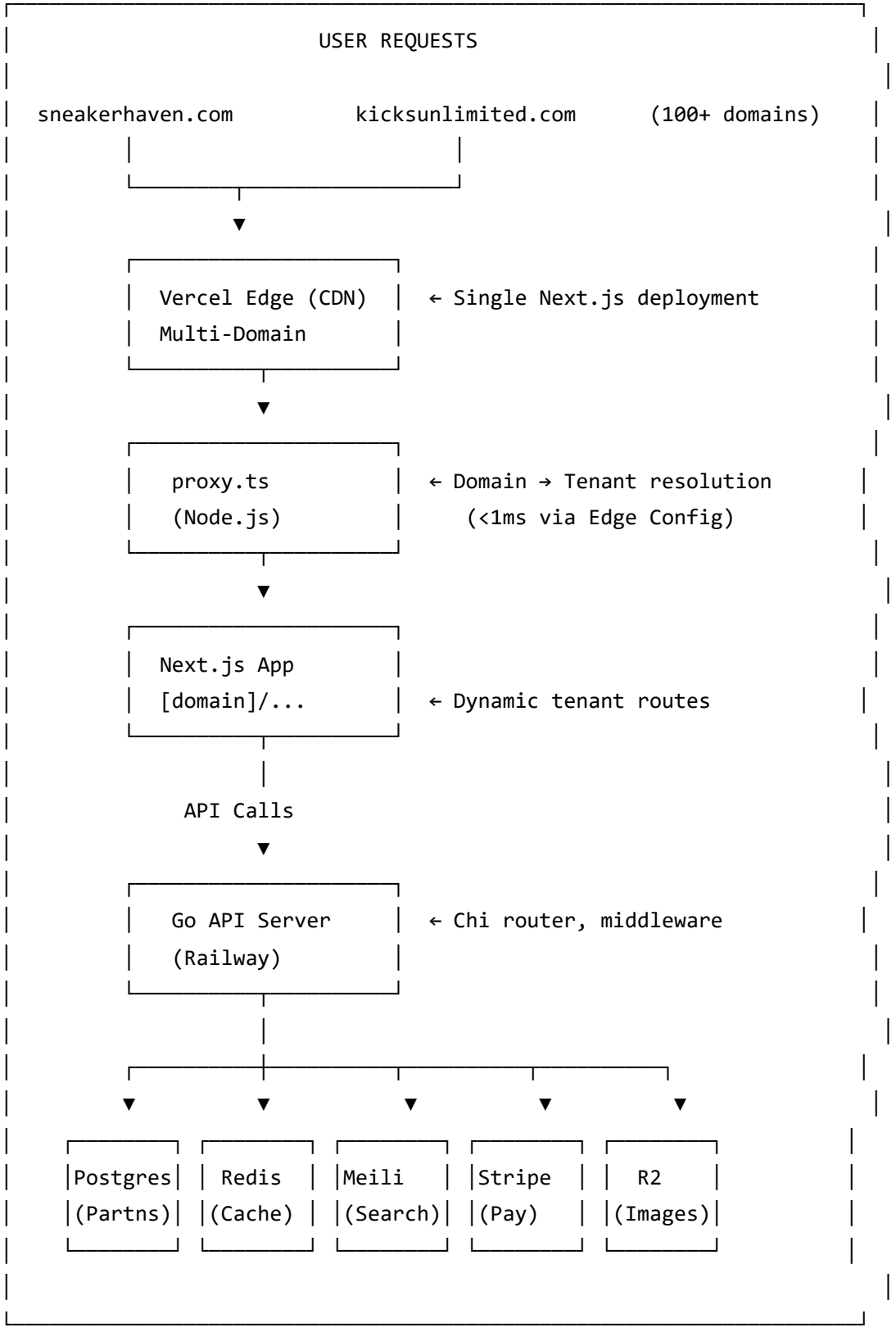
Component	Technology	Version	Why
Framework	Next.js	16	App Router, proxy.ts for multi-domain
Language	TypeScript	5.1+	Type safety, better DX
Styling	Tailwind CSS	3.4+	Utility-first, fast development
Components	shadcn/ui	Latest	Accessible, customizable components
Forms	React Hook Form	Latest	Performant form handling
State	Zustand	Latest	Lightweight state management
API Client	Fetch + SWR	Latest	Data fetching with caching
Payments	Stripe Elements	Latest	Embedded payment forms

Infrastructure

Service	Provider	Plan	Cost/Month
Hosting (API)	Railway	Variable	$10-90
Hosting (Frontend)	Vercel	Pro	$20
Database	Railway	Managed PG	$15-90
Search	Railway	Self-hosted	$8-35
CDN/Storage	Cloudflare	R2 + CDN	$1-12
Cache	Upstash	Fixed 1GB	$20-100
Auth	Clerk	Pay-as-you-go	$0-900
Domain	Namecheap	.com	$1/month

Architecture Overview

High-Level System Design



Multi-Domain Routing Flow

1. User visits: sneakerhaven.com/products
   ▼
2. DNS resolves to Vercel edge location (global CDN)
   ▼
3. proxy.ts extracts hostname: "sneakerhaven.com"
   ▼
4. Looks up tenant in Edge Config (<1ms):
   ```
   {
     "tenant:sneakerhaven.com": {
       "id": "01HQST1...",
       "slug": "sneaker-haven",
       "theme": { ... }
     }
   }
   ```
   ▼
5. Rewrites URL internally:
   sneakerhaven.com/products → /sneakerhaven.com/products
   ▼
6. Next.js routes to: app/[domain]/products/page.tsx
   ▼
7. Page fetches products from Go API:
   GET /v1/products?tenant_id=01HQST1...
   ▼
8. API queries Postgres partition for tenant 01HQST1 only
   ▼
9. Returns products, page renders with tenant theme
   ▼
10. User sees: Sneaker Haven branded product listing

Tenant Isolation Strategy

Database Level:

```
-- Table partitioning by tenant_id
CREATE TABLE products (...) PARTITION BY LIST (tenant_id);

-- Each tenant gets dedicated partition
CREATE TABLE products_tenant_01HQST1 PARTITION OF products
FOR VALUES IN ('01HQST1AAAAAAA1111111111');

-- RLS policies as backup
CREATE POLICY tenant_isolation ON products
  USING (tenant_id = current_setting('app.current_tenant_id')::ULID);
```

Application Level:

```
// Every query MUST filter by tenant_id
func (r *ProductRepository) GetProducts(ctx context.Context, tenantID ulid.ULID) {
    query := `SELECT * FROM products WHERE tenant_id = $1`
    // Partition pruning happens automatically
}
```

Cache Level:

```
// Cache keys include tenant_id
cacheKey := fmt.Sprintf("products:%s:listing", tenantID)
```

Storage Level:

```
R2 bucket structure:
/production/
  /01HQST1.../products/abc/image.webp  ← Tenant 1
  /01HQST2.../products/xyz/image.webp  ← Tenant 2
```

Complete Cost Analysis

Production Environment (Launch - 5 Tenants)

Service	Configuration	Monthly Cost	Notes
Railway - PostgreSQL	2 vCPU, 2GB RAM, 10GB	$15	Partitioned tables
Railway - Go API	Shared vCPU, ~512MB	$10	Auto-scaling
Railway - Meilisearch	Shared vCPU, 1GB RAM	$8	Self-hosted search
Vercel Pro	Unlimited bandwidth	$20	Required for custom domains
Cloudflare R2	10-25GB storage	$1	Product images
Upstash Redis	Fixed 1GB, 40M cmd/mo	$20	Rate limiting, cache
Clerk	<10K MAU	$0	Auth, free tier
AWS SES	~5K emails/month	$2	Transactional email
Cloudflare Bot	Free tier	$0	DDoS protection
Domain	rdk.com	$1	Annual/12
Vercel Edge Config	100 tenants	$10	<1ms tenant lookups
TOTAL PRODUCTION		**$87/month**	

Staging Environment (Optimized)

Service	Configuration	Monthly Cost	Strategy
Railway - PostgreSQL	Starter, 256MB, 1GB	$5	Small dedicated instance
Railway - Go API	Free tier, auto-sleep	$0	$5 credit covers usage

Service	Configuration	Monthly Cost	Strategy
Meilisearch	Shared from production	$0	Separate index
Vercel	Same project	$0	Staging branch
Cloudflare R2	staging/ prefix	$0	Shared bucket
Upstash Redis	DB 1	$0	Separate database
Clerk	Staging org	$0	Within free tier
Email	Mailgun free tier	$0	5K emails/month
TOTAL STAGING		**$5/month**	94% cost reduction!

Cost Scaling Projection

Phase	Tenants	MAU	Production	Staging	Total	Margin
Launch	1-5	1K	$87	$5	**$92**	61%
Growth	10-20	25K	$109	$5	**$114**	82%
Scale	30-50	100K	$304	$5	**$309**	75%
Target	100+	500K	$1,348	$5	**$1,353**	74%

Revenue Projections (Conservative)

Phase	Tenants	Avg Price	MRR	Infrastructure	Net Profit	Margin
Month 1	1	$29	$29	$92	-$63	-217%
Month 3	5	$29	$145	$92	$53	37%
Month 6	12	$35	$420	$114	$306	73%
Month 12	25	$39	$975	$180	$795	82%
Month 18	40	$43	$1,720	$240	$1,480	86%
Month 24	60	$45	$2,700	$309	$2,391	89%

Break-Even: Month 3 (5 paying tenants)

2-Year Net Profit: ~$28,000

Pricing Tiers

```
┌─────────────────────────────────────────────────────┐
│ STARTER PLAN - $29/month                            │
├─────────────────────────────────────────────────────┤
│ • 500 products                                      │
│ • 100 orders/month                                  │
│ • 5GB storage                                       │
│ • Email support (48hr response)                     │
│ • Basic component templates (10 options)            │
│ • Standard shipping rates                           │
│ • Stripe integration                                │
│ • SSL certificate included                          │
└─────────────────────────────────────────────────────┘
```

```
┌─────────────────────────────────────────────────────┐
│ PRO PLAN - $49/month                                │
├─────────────────────────────────────────────────────┤
│ • 2,000 products                                    │
│ • 500 orders/month                                  │
│ • 20GB storage                                      │
│ • Priority support (24hr response)                  │
│ • Premium templates (all 20+ options)               │
│ • Custom CSS per component                          │
│ • Advanced analytics                                │
│ • Bulk product import                               │
└─────────────────────────────────────────────────────┘
```

```
┌─────────────────────────────────────────────────────┐
│ ENTERPRISE PLAN - Custom Pricing                    │
├─────────────────────────────────────────────────────┤
│ • Unlimited products & orders                       │
│ • 100GB+ storage                                    │
│ • Dedicated support (4hr SLA)                       │
│ • Custom component development                      │
│ • API access                                        │
│ • White-label mobile app                            │
│ • Multi-user team accounts                          │
│ • Priority processing                               │
└─────────────────────────────────────────────────────┘
```

Database Schema

Core Tables

tenants

```
CREATE TABLE tenants (
  -- Identity
  id ULID PRIMARY KEY,
  slug VARCHAR(100) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL,

  -- Clerk integration
  clerk_org_id VARCHAR(255) UNIQUE NOT NULL,

  -- Branding
  logo_url TEXT,
  favicon_url TEXT,
  primary_color VARCHAR(7) NOT NULL DEFAULT '#000000',
  secondary_color VARCHAR(7) NOT NULL DEFAULT '#FFFFFF',

  -- Theme configuration (JSONB)
  theme_config JSONB NOT NULL DEFAULT '{}'::jsonb,

  -- Subscription
  plan_tier VARCHAR(50) NOT NULL DEFAULT 'starter',
  max_products INTEGER NOT NULL DEFAULT 500,
  max_monthly_orders INTEGER NOT NULL DEFAULT 100,
  max_storage_gb INTEGER NOT NULL DEFAULT 5,

  -- Stripe Connect
  stripe_account_id VARCHAR(255),
  stripe_account_status VARCHAR(50) DEFAULT 'not_connected',
  stripe_onboarding_completed_at TIMESTAMPTZ,

  -- Tax settings
  tax_enabled BOOLEAN DEFAULT false,
  tax_id VARCHAR(100),
  tax_nexus JSONB DEFAULT '[]'::jsonb,

  -- Contact
  support_email VARCHAR(255) NOT NULL,
```

```
  support_phone VARCHAR(50),

  -- Status
  status VARCHAR(50) NOT NULL DEFAULT 'active',

  -- Timestamps
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at TIMESTAMPTZ
);

CREATE INDEX idx_tenants_slug ON tenants(slug);
CREATE INDEX idx_tenants_clerk_org ON tenants(clerk_org_id);
CREATE INDEX idx_tenants_status ON tenants(status) WHERE deleted_at IS NULL;
```

domains (Custom domain management)

```
CREATE TABLE domains (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

  domain VARCHAR(255) UNIQUE NOT NULL,
  is_primary BOOLEAN NOT NULL DEFAULT false,

  -- Verification
  status VARCHAR(50) NOT NULL DEFAULT 'pending_verification',
  dns_config JSONB NOT NULL DEFAULT '{}'::jsonb,
  verified_at TIMESTAMPTZ,

  -- SSL
  ssl_status VARCHAR(50) DEFAULT 'pending',
  ssl_issued_at TIMESTAMPTZ,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_domains_tenant ON domains(tenant_id);
CREATE INDEX idx_domains_domain ON domains(domain);
CREATE INDEX idx_domains_status ON domains(status);
CREATE UNIQUE INDEX idx_domains_primary ON domains(tenant_id, is_primary)
WHERE is_primary = true;
```

products (Partitioned by tenant_id)

```
CREATE TABLE products (
  id ULID NOT NULL,
  tenant_id ULID NOT NULL,

  -- Product info
  slug VARCHAR(255) NOT NULL,
  title VARCHAR(255) NOT NULL,
  brand VARCHAR(100),
  model VARCHAR(100),
  colorway VARCHAR(100),
  description TEXT,

  -- Pricing
  base_price_cents INTEGER NOT NULL,
  compare_at_price_cents INTEGER,

  -- Inventory
  total_quantity INTEGER NOT NULL DEFAULT 0,
  reserved_quantity INTEGER NOT NULL DEFAULT 0,

  -- Status
  status VARCHAR(50) NOT NULL DEFAULT 'draft',

  -- SEO
  meta_title VARCHAR(255),
  meta_description TEXT,

  -- Search (for Postgres FTS fallback)
  search_vector tsvector,

  -- Timestamps
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at TIMESTAMPTZ,

  -- Partition key MUST be part of primary key
  PRIMARY KEY (tenant_id, id),
  UNIQUE (tenant_id, slug)
) PARTITION BY LIST (tenant_id);

-- Default partition for new tenants
```

```sql
CREATE TABLE products_default PARTITION OF products DEFAULT;

-- Indexes (will be created per partition)
CREATE INDEX idx_products_listing ON products (
  tenant_id, status, created_at DESC
) INCLUDE (id, title, brand, base_price_cents, total_quantity)
WHERE deleted_at IS NULL;

CREATE INDEX idx_products_brand ON products (
  tenant_id, brand, status
) INCLUDE (id, title, base_price_cents)
WHERE deleted_at IS NULL;

CREATE INDEX idx_products_search ON products
USING GIN(search_vector)
WHERE deleted_at IS NULL;
```

product_variants (Partitioned by tenant_id)

```sql
CREATE TABLE product_variants (
  id ULID NOT NULL,
  tenant_id ULID NOT NULL,
  product_id ULID NOT NULL,

  -- Variant specifics
  sku VARCHAR(100) NOT NULL,
  size VARCHAR(20) NOT NULL,
  size_system VARCHAR(10) NOT NULL DEFAULT 'US',
  condition VARCHAR(50) NOT NULL DEFAULT 'new',

  -- Inventory
  quantity INTEGER NOT NULL DEFAULT 0,
  reserved_quantity INTEGER NOT NULL DEFAULT 0,

  -- Pricing override (optional)
  price_cents INTEGER,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at TIMESTAMPTZ,

  PRIMARY KEY (tenant_id, id),
  UNIQUE (tenant_id, sku),
  UNIQUE (tenant_id, product_id, size, condition)
) PARTITION BY LIST (tenant_id);

CREATE TABLE product_variants_default PARTITION OF product_variants DEFAULT;

CREATE INDEX idx_variants_product ON product_variants (
  tenant_id, product_id
) INCLUDE (id, sku, size, quantity, reserved_quantity)
WHERE deleted_at IS NULL;

CREATE INDEX idx_variants_sku ON product_variants (tenant_id, sku)
WHERE deleted_at IS NULL;
```

product_images

```sql
CREATE TABLE product_images (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL,
  product_id ULID NOT NULL,

  url TEXT NOT NULL,
  position INTEGER NOT NULL DEFAULT 0,
  alt_text VARCHAR(255),

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_product_images ON product_images(tenant_id, product_id, position);
```

orders (Partitioned by tenant_id)

```
CREATE TABLE orders (
  id ULID NOT NULL,
  tenant_id ULID NOT NULL,

  -- Order identity
  order_number VARCHAR(50) NOT NULL,

  -- Customer (authenticated or guest)
  user_id ULID,
  guest_email VARCHAR(255),
  guest_token_hash VARCHAR(255),
  guest_token_expires_at TIMESTAMPTZ,

  -- Pricing
  subtotal_cents INTEGER NOT NULL,
  tax_cents INTEGER NOT NULL DEFAULT 0,
  shipping_cents INTEGER NOT NULL DEFAULT 0,
  total_cents INTEGER NOT NULL,

  -- Stripe
  stripe_payment_intent_id VARCHAR(255),
  stripe_charge_id VARCHAR(255),

  -- Shipping
  shipping_method VARCHAR(100),
  shipping_address JSONB NOT NULL,
  billing_address JSONB NOT NULL,
  tracking_number VARCHAR(255),
  tracking_carrier VARCHAR(100),

  -- Status
  status VARCHAR(50) NOT NULL DEFAULT 'pending',

  -- Fulfillment
  fulfilled_at TIMESTAMPTZ,
  shipped_at TIMESTAMPTZ,
  delivered_at TIMESTAMPTZ,
  cancelled_at TIMESTAMPTZ,
  refunded_at TIMESTAMPTZ,

  -- Notes
```

```
  customer_notes TEXT,
  internal_notes TEXT,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at TIMESTAMPTZ,

  PRIMARY KEY (tenant_id, id),
  UNIQUE (tenant_id, order_number)
) PARTITION BY LIST (tenant_id);

CREATE TABLE orders_default PARTITION OF orders DEFAULT;

CREATE INDEX idx_orders_customer ON orders (
  tenant_id, user_id, created_at DESC
) INCLUDE (id, order_number, status, total_cents)
WHERE deleted_at IS NULL;

CREATE INDEX idx_orders_guest ON orders (
  tenant_id, guest_email, created_at DESC
) WHERE user_id IS NULL AND deleted_at IS NULL;

CREATE INDEX idx_orders_status ON orders (
  tenant_id, status, created_at DESC
) WHERE deleted_at IS NULL;

CREATE INDEX idx_orders_number ON orders (tenant_id, order_number)
WHERE deleted_at IS NULL;
```

order_items (Partitioned by tenant_id)

```sql
CREATE TABLE order_items (
  id ULID NOT NULL,
  tenant_id ULID NOT NULL,
  order_id ULID NOT NULL,
  product_id ULID NOT NULL,
  variant_id ULID,

  -- Snapshot at purchase time
  product_title VARCHAR(255) NOT NULL,
  product_brand VARCHAR(100),
  variant_size VARCHAR(20),
  variant_sku VARCHAR(100),

  quantity INTEGER NOT NULL DEFAULT 1,
  unit_price_cents INTEGER NOT NULL,
  total_price_cents INTEGER NOT NULL,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  PRIMARY KEY (tenant_id, id)
) PARTITION BY LIST (tenant_id);

CREATE TABLE order_items_default PARTITION OF order_items DEFAULT;

CREATE INDEX idx_order_items ON order_items(tenant_id, order_id);
```

component_templates

```
CREATE TABLE component_templates (
  id ULID PRIMARY KEY,

  name VARCHAR(100) NOT NULL,
  category VARCHAR(50) NOT NULL,
  description TEXT,
  preview_image_url TEXT,

  -- JSON Schema for configuration
  config_schema JSONB NOT NULL,

  -- Premium feature
  is_premium BOOLEAN DEFAULT false,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_component_templates ON component_templates(category);
```

tenant_components

```
CREATE TABLE tenant_components (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  template_id ULID NOT NULL REFERENCES component_templates(id),

  page_location VARCHAR(50) NOT NULL,
  position INTEGER NOT NULL DEFAULT 0,

  -- Custom configuration (overrides template defaults)
  custom_config JSONB NOT NULL DEFAULT '{}'::jsonb,
  custom_css TEXT,

  is_active BOOLEAN DEFAULT true,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_tenant_components ON tenant_components(tenant_id, page_location, position);
```

Database Stats (at target scale)

```
100 Tenants × 5,000 Products = 500,000 products
100 Partitions (products_tenant_*)

Per partition:
- ~5,000 rows
- ~15 MB storage
- Queries scan only 1 partition (5K rows vs 500K)

Total database size: ~2.5 GB
Query performance: <50ms P95
```

API Design

API Architecture

Base URL: `https://api.rdk.com/v1`

Authentication: Clerk JWT in `Authorization: Bearer <token>` header

Tenant Context: Extracted from Clerk organization ID

Response Format:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "request_id": "req_abc123",
    "tenant_id": "01HQST...",
    "timestamp": "2026-02-09T12:00:00Z"
  },
  "pagination": {
    "page": 1,
    "per_page": 20,
    "total_items": 150,
    "total_pages": 8
  }
}
```

Public Storefront API

Products

```
GET /v1/store/products
Query: page, per_page, brand, min_price, max_price, size, sort

Response:
{
  "success": true,
  "data": [
    {
      "id": "01HQST...",
      "slug": "air-jordan-1-bred",
      "title": "Air Jordan 1 Retro High OG 'Bred'",
      "brand": "Nike",
      "base_price_cents": 17000,
      "images": [
        {
          "url": "https://images.rdk.com/.../1.webp",
          "alt": "Air Jordan 1 front view"
        }
      ],
      "variants": [
        {
          "id": "01HQST...",
          "size": "10",
          "quantity": 3,
          "available": true
        }
      ]
    }
  ],
  "pagination": { ... }
}
```

```
GET /v1/store/products/:slug

Response:
{
  "success": true,
  "data": {
    "id": "01HQST...",
    "slug": "air-jordan-1-bred",
    "title": "Air Jordan 1 Retro High OG 'Bred'",
    "description": "The iconic colorway...",
    "brand": "Nike",
    "model": "Air Jordan 1",
    "colorway": "Bred",
    "base_price_cents": 17000,
    "images": [...],
    "variants": [...],
    "related_products": [...]
  }
}
```

Search

```
GET /v1/store/search
Query: q (search query), page, per_page

Uses Meilisearch with Postgres FTS fallback

Response:
{
  "success": true,
  "data": [
    { /* product */ }
  ],
  "search_meta": {
    "query": "jordan red",
    "processing_time_ms": 12,
    "total_hits": 45,
    "engine": "meilisearch"
  }
}
```

Cart Validation

```
POST /v1/store/cart/validate
Body:
{
  "items": [
    {
      "variant_id": "01HQST...",
      "quantity": 2
    }
  ]
}

Response:
{
  "success": true,
  "data": {
    "valid": true,
    "items": [
      {
        "variant_id": "01HQST...",
        "available_quantity": 3,
        "is_available": true,
        "unit_price_cents": 17000
      }
    ],
    "subtotal_cents": 34000
  }
}
```

Checkout API

```
POST /v1/checkout/session
Body:
{
  "items": [
    {
      "variant_id": "01HQST...",
      "quantity": 1
    }
  ],
  "shipping_address": { ... },
  "billing_address": { ... },
  "shipping_method": "standard"
}

Response:
{
  "success": true,
  "data": {
    "checkout_session_id": "cs_...",
    "stripe_session_url": "https://checkout.stripe.com/...",
    "expires_at": "2026-02-09T13:00:00Z"
  }
}
```

```
POST /v1/checkout/confirm
Body:
{
  "stripe_session_id": "cs_...",
  "guest_email": "customer@example.com" // If not authenticated
}

Response:
{
  "success": true,
  "data": {
    "order_id": "01HQST...",
    "order_number": "ORD-1001",
    "guest_access_token": "eyJ..." // If guest checkout
  }
}
```

Orders API

```
GET /v1/orders
Query: status, page, per_page
Auth: Required (customer sees only their orders)

Response:
{
  "success": true,
  "data": [
    {
      "id": "01HQST...",
      "order_number": "ORD-1001",
      "status": "shipped",
      "total_cents": 18500,
      "tracking_number": "1Z999...",
      "created_at": "2026-02-01T10:00:00Z"
    }
  ]
}
```

```
GET /v1/orders/:order_number
Query: guest_token (if guest order)

Response:
{
  "success": true,
  "data": {
    "id": "01HQST...",
    "order_number": "ORD-1001",
    "status": "shipped",
    "items": [
      {
        "product_title": "Air Jordan 1 Bred",
        "variant_size": "10",
        "quantity": 1,
        "unit_price_cents": 17000
      }
    ],
    "subtotal_cents": 17000,
    "tax_cents": 1200,
    "shipping_cents": 300,
    "total_cents": 18500,
    "shipping_address": { ... },
    "tracking_number": "1Z999...",
    "tracking_carrier": "UPS",
    "tracking_url": "https://ups.com/track/...",
    "timeline": [
      {
        "status": "created",
        "timestamp": "2026-02-01T10:00:00Z"
      },
      {
        "status": "paid",
        "timestamp": "2026-02-01T10:01:00Z"
      },
      {
        "status": "shipped",
        "timestamp": "2026-02-02T14:30:00Z"
      }
    ]
  }
}
```

Admin API (requires admin role)

Products

```
POST /v1/admin/products
Body:
{
  "title": "Air Jordan 1 Bred",
  "brand": "Nike",
  "description": "...",
  "base_price_cents": 17000,
  "variants": [
    {
      "size": "10",
      "sku": "554770-065-10",
      "quantity": 5
    }
  ],
  "images": [
    "https://temp-upload.rdk.com/img1.jpg"
  ]
}

Response:
{
  "success": true,
  "data": {
    "id": "01HQST...",
    "slug": "air-jordan-1-bred"
  }
}


PATCH /v1/admin/products/:id
PUT /v1/admin/products/:id/images
DELETE /v1/admin/products/:id
```

Orders

```
GET /v1/admin/orders
Query: status, search, date_from, date_to, page

PATCH /v1/admin/orders/:id/fulfill
Body:
{
  "tracking_number": "1Z999...",
  "tracking_carrier": "UPS"
}

POST /v1/admin/orders/:id/refund
Body:
{
  "amount_cents": 18500, // Full or partial
  "reason": "customer_request"
}
```

Settings

```
GET /v1/admin/settings
PATCH /v1/admin/settings/general
PATCH /v1/admin/settings/theme
PATCH /v1/admin/settings/components
```

Domains

```
POST /v1/admin/domains
Body:
{
  "domain": "sneakerhaven.com"
}

Response:
{
  "success": true,
  "data": {
    "domain": "sneakerhaven.com",
    "status": "pending_verification",
    "dns_config": {
      "type": "A",
      "name": "@",
      "value": "76.76.21.21"
    }
  }
}

POST /v1/admin/domains/:domain/verify
DELETE /v1/admin/domains/:domain
```

Stripe Connect

```
POST /v1/admin/stripe/connect/onboard

Response:
{
  "success": true,
  "data": {
    "onboarding_url": "https://connect.stripe.com/..."
  }
}

GET /v1/admin/stripe/account

Response:
{
  "success": true,
  "data": {
    "account_id": "acct_...",
    "charges_enabled": true,
    "payouts_enabled": true,
    "requirements": []
  }
}
```

Webhooks

```
POST /v1/webhooks/stripe
- Handles Stripe events
- Verifies signature
- Processes payment_intent.succeeded, etc.

POST /v1/webhooks/clerk
- Syncs users and organizations
- Creates tenants from Clerk orgs

POST /v1/webhooks/shippo
- Updates tracking information
```

Frontend Architecture

Next.js 16 Features Used

1. proxy.ts (replaces middleware.ts)

- Runs on Node.js runtime (not Edge)
- Extracts hostname for multi-domain routing
- <1ms latency with Edge Config caching

2. App Router with Dynamic Routes

- `app/[domain]/...` for tenant storefronts
- `app/admin/...` for admin dashboard
- Server Components by default
- Client Components when needed

3. Server Actions

- Form submissions
- Cart operations
- Admin actions

4. Streaming & Suspense

- Loading states
- Progressive rendering
- Skeleton screens

Theme System

Tenant Theme Configuration (JSONB):

```
{
  "colors": {
    "primary": "#FF4500",
    "secondary": "#1E90FF",
    "accent": "#FFD700",
    "background": "#FFFFFF",
    "text": "#1A1A1A"
  },
  "typography": {
    "headingFont": "Montserrat",
    "bodyFont": "Inter",
    "scale": "comfortable"
  },
  "components": {
    "homepage_hero": {
      "template": "hero_video",
      "config": {
        "videoUrl": "https://videos.rdk.com/.../hero.mp4",
        "heading": "Rare Kicks, Unbeatable Prices",
        "ctaText": "Shop Now"
      }
    },
    "product_grid": {
      "template": "grid_masonry",
      "config": {
        "columns": 4,
        "gap": "md"
      }
    }
  },
  "customCSS": "/* Optional custom styles */"
}
```

CSS Generation:

```
// lib/utils/theme.ts
export function generateThemeCSS(theme: ThemeConfig): string {
  return `
    :root {
      --color-primary: ${theme.colors.primary};
      --color-secondary: ${theme.colors.secondary};
      --font-heading: ${theme.typography.headingFont};
      --font-body: ${theme.typography.bodyFont};
    }
    ${theme.customCSS || ''}
  `;
}
```

Component Templates

Available Templates:

Hero (5 options):

- HeroVideo: Full-screen video background
- HeroCarousel: Image slideshow
- HeroSplit: Image left, text right
- HeroMinimal: Text-only centered
- HeroAnimated: Motion graphics

Navigation (3 options):

- NavSticky: Fixed header
- NavMega: Dropdown mega menu
- NavSidebar: Mobile-first sidebar

Product Grid (4 options):

- GridMasonry: Pinterest-style
- GridStandard: Traditional rows
- GridCarousel: Horizontal scroll
- GridList: List view with details

Footer (3 options):

- FooterMinimal: Copyright + links
- FooterExpanded: Multi-column

- FooterNewsletter: With signup form

Backend Folder Structure

```
backend/
│
├── cmd/                                  # Entry points
│   ├── api/
│   │   └── main.go                       # HTTP server entry
│   ├── migrate/
│   │   └── main.go                       # Migration runner
│   └── worker/
│       └── main.go                       # Background worker
│
├── internal/                             # Private application code
│   │
│   ├── api/                              # HTTP layer
│   │   │
│   │   ├── handlers/                     # Request handlers
│   │   │   ├── health.go                 # Health check
│   │   │   ├── auth.go                   # Auth callbacks
│   │   │   ├── tenants.go                # Tenant CRUD
│   │   │   ├── products.go               # Product endpoints
│   │   │   ├── search.go                 # Search endpoint
│   │   │   ├── cart.go                   # Cart validation
│   │   │   ├── checkout.go               # Checkout session
│   │   │   ├── orders.go                 # Order queries
│   │   │   ├── domains.go                # Domain management
│   │   │   ├── webhooks.go               # Webhook receivers
│   │   │   └── admin.go                  # Admin operations
│   │   │
│   │   ├── middleware/                   # HTTP middleware
│   │   │   ├── request_id.go             # Request ID
│   │   │   ├── logger.go                 # Request logging
│   │   │   ├── cors.go                   # CORS headers
│   │   │   ├── tenant_context.go         # Tenant injection
│   │   │   ├── rate_limit.go             # Rate limiting
│   │   │   ├── auth.go                   # Clerk JWT validation
│   │   │   └── rbac.go                   # Role check
│   │   │
│   │   ├── router.go                     # Chi router setup
│   │   ├── server.go                     # HTTP server config
│   │   └── responses.go                  # Response helpers
│   │
```

```
│   ├── services/                       # Business logic
│   │   ├── tenant_service.go           # Tenant operations
│   │   ├── product_service.go          # Product logic
│   │   ├── order_service.go            # Order processing
│   │   ├── checkout_service.go         # Checkout flow
│   │   ├── inventory_service.go        # Stock management
│   │   ├── search_service.go           # Meilisearch + Postgres FTS
│   │   ├── domain_service.go           # Vercel Domains API
│   │   ├── email_service.go            # AWS SES emails
│   │   ├── cache_service.go            # Redis operations
│   │   ├── storage_service.go          # R2 uploads
│   │   ├── stripe_service.go           # Stripe integration
│   │   └── webhook_service.go          # Webhook processing
│   │
│   ├── repositories/                   # Data access
│   │   ├── base_repository.go          # Base repo with metrics
│   │   ├── tenant_repo.go
│   │   ├── product_repo.go
│   │   ├── variant_repo.go
│   │   ├── order_repo.go
│   │   ├── order_item_repo.go
│   │   ├── user_repo.go
│   │   ├── component_repo.go
│   │   └── domain_repo.go
│   │
│   ├── models/                         # Domain models
│   │   ├── tenant.go
│   │   ├── product.go
│   │   ├── variant.go
│   │   ├── order.go
│   │   ├── order_item.go
│   │   ├── user.go
│   │   ├── component.go
│   │   └── domain.go
│   │
│   ├── jobs/                           # Background jobs (Asynq)
│   │   ├── product_index_job.go        # Index in Meilisearch
│   │   ├── order_email_job.go          # Send order emails
│   │   ├── stripe_webhook_job.go       # Process webhooks
│   │   ├── domain_verify_job.go        # Check DNS status
│   │   └── cleanup_job.go              # Delete old data
│   │
│   └── config/                         # Configuration
```

```
│       ├── config.go                    # Load env vars
│       ├── database.go                  # DB connection
│       ├── redis.go                     # Redis connection
│       ├── clerk.go                     # Clerk SDK
│       ├── stripe.go                    # Stripe SDK
│       ├── meilisearch.go               # Meilisearch client
│       ├── vercel.go                    # Vercel API client
│       └── aws.go                       # AWS SDK (SES)
│
├── pkg/                                 # Public packages
│   ├── logger/
│   │   └── logger.go                    # Structured logging
│   ├── validator/
│   │   └── validator.go                 # Input validation
│   ├── errors/
│   │   └── errors.go                    # Error types
│   └── ulid/
│       └── ulid.go                      # ULID generation
│
├── migrations/                          # SQL migrations (Goose)
│   ├── 00001_create_tenants.sql
│   ├── 00002_create_users.sql
│   ├── 00003_create_products.sql
│   ├── 00004_create_variants.sql
│   ├── 00005_create_orders.sql
│   ├── 00006_create_order_items.sql
│   ├── 00007_create_domains.sql
│   ├── 00008_create_components.sql
│   ├── 00009_add_rls_policies.sql
│   ├── 00010_partition_products.sql
│   ├── 00011_partition_variants.sql
│   ├── 00012_partition_orders.sql
│   ├── 00013_create_indexes.sql
│   └── 00014_seed_component_templates.sql
│
├── seeds/                               # Test data
│   ├── dev/
│   │   ├── tenants.sql                  # 2 example tenants
│   │   ├── products.sql                 # Sample products
│   │   └── domains.sql                  # Domain mappings
│   └── staging/
│       └── minimal.sql                  # Minimal staging data
│
```

```
├── tests/                              # Tests
│   ├── unit/
│   │   ├── services/
│   │   ├── repositories/
│   │   └── models/
│   ├── integration/
│   │   ├── api/
│   │   ├── database/
│   │   └── webhooks/
│   └── e2e/
│       └── flows/
│           ├── checkout_test.go
│           └── order_test.go
│
├── scripts/                            # Utility scripts
│   ├── dev-setup.sh                    # Setup local dev
│   ├── seed-dev-data.sh                # Seed database
│   ├── create-migration.sh             # New migration
│   ├── rollback-migration.sh           # Rollback migration
│   ├── create-tenant.sh                # Create new tenant
│   └── clean-staging.sh                # Clean staging data
│
├── docs/                               # Documentation
│   ├── API.md                          # API reference
│   ├── ARCHITECTURE.md                 # Architecture docs
│   ├── DEPLOYMENT.md                   # Deployment guide
│   └── DEVELOPMENT.md                  # Dev setup
│
├── Dockerfile                          # Multi-stage build
├── go.mod                              # Go dependencies
├── go.sum                              # Go checksums
├── .env.example                        # Example env vars
├── Makefile                            # Common commands
└── README.md                           # Backend README
```

Backend File Count: ~80 files

Frontend Folder Structure

```
frontend/
│
├── public/                                  # Static files
│   ├── fonts/
│   │   ├── inter/
│   │   ├── montserrat/
│   │   └── playfair/
│   ├── images/
│   │   ├── placeholders/
│   │   └── icons/
│   └── favicon.ico
│
├── src/
│   │
│   ├── app/                                 # Next.js App Router
│   │   │
│   │   ├── [domain]/                        # Tenant routes
│   │   │   ├── layout.tsx                   # Tenant layout
│   │   │   ├── page.tsx                     # Homepage
│   │   │   ├── products/
│   │   │   │   ├── page.tsx                 # Product list
│   │   │   │   └── [slug]/
│   │   │   │       └── page.tsx             # Product detail
│   │   │   ├── cart/
│   │   │   │   └── page.tsx
│   │   │   ├── checkout/
│   │   │   │   ├── page.tsx
│   │   │   │   └── success/
│   │   │   │       └── page.tsx
│   │   │   ├── account/
│   │   │   │   ├── layout.tsx
│   │   │   │   ├── page.tsx
│   │   │   │   └── orders/
│   │   │   │       ├── page.tsx
│   │   │   │       └── [id]/
│   │   │   │           └── page.tsx
│   │   │   ├── about/
│   │   │   │   └── page.tsx
│   │   │   ├── contact/
│   │   │   │   └── page.tsx
```

```
│   │   │   └── (policies)/             # Route group
│   │   │       ├── privacy/
│   │   │       ├── terms/
│   │   │       ├── shipping/
│   │   │       └── returns/
│   │   │
│   │   ├── admin/                       # Admin dashboard
│   │   │   ├── layout.tsx               # Admin layout
│   │   │   ├── page.tsx                 # Dashboard
│   │   │   ├── products/
│   │   │   │   ├── page.tsx             # Product list
│   │   │   │   ├── new/
│   │   │   │   │   └── page.tsx
│   │   │   │   └── [id]/
│   │   │   │       └── edit/
│   │   │   │           └── page.tsx
│   │   │   ├── orders/
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/
│   │   │   │       └── page.tsx
│   │   │   ├── analytics/
│   │   │   │   └── page.tsx
│   │   │   └── settings/
│   │   │       ├── page.tsx
│   │   │       ├── general/
│   │   │       │   └── page.tsx
│   │   │       ├── domain/
│   │   │       │   └── page.tsx         # Domain management
│   │   │       ├── theme/
│   │   │       │   └── page.tsx         # Theme customizer
│   │   │       ├── components/
│   │   │       │   └── page.tsx         # Component selector
│   │   │       ├── tax/
│   │   │       │   └── page.tsx         # Stripe Tax settings
│   │   │       └── payments/
│   │   │           └── page.tsx         # Stripe Connect
│   │   │
│   │   ├── api/                         # API routes
│   │   │   ├── clerk-webhook/
│   │   │   │   └── route.ts
│   │   │   ├── revalidate/
│   │   │   │   └── route.ts
│   │   │   └── health/
```

```
│   │   │       └── route.ts
│   │   │
│   │   ├── layout.tsx                    # Root layout
│   │   ├── globals.css
│   │   └── not-found.tsx
│   │
│   ├── components/
│   │   │
│   │   ├── templates/                    # Customizable templates
│   │   │   ├── hero/
│   │   │   │   ├── HeroVideo.tsx
│   │   │   │   ├── HeroCarousel.tsx
│   │   │   │   ├── HeroSplit.tsx
│   │   │   │   ├── HeroMinimal.tsx
│   │   │   │   └── HeroAnimated.tsx
│   │   │   ├── navigation/
│   │   │   │   ├── NavSticky.tsx
│   │   │   │   ├── NavMega.tsx
│   │   │   │   └── NavSidebar.tsx
│   │   │   ├── product-grid/
│   │   │   │   ├── GridMasonry.tsx
│   │   │   │   ├── GridStandard.tsx
│   │   │   │   ├── GridCarousel.tsx
│   │   │   │   └── GridList.tsx
│   │   │   └── footer/
│   │   │       ├── FooterMinimal.tsx
│   │   │       ├── FooterExpanded.tsx
│   │   │       └── FooterNewsletter.tsx
│   │   │
│   │   ├── base/                         # Base UI components
│   │   │   ├── Button.tsx
│   │   │   ├── Input.tsx
│   │   │   ├── Card.tsx
│   │   │   ├── Modal.tsx
│   │   │   └── (12 more...)
│   │   │
│   │   ├── shared/                       # Shared components
│   │   │   ├── ProductCard.tsx
│   │   │   ├── CartItem.tsx
│   │   │   ├── SearchBar.tsx
│   │   │   ├── FilterSidebar.tsx
│   │   │   └── (15 more...)
│   │   │
```

```
│   │   ├── admin/                      # Admin components
│   │   │   ├── AdminSidebar.tsx
│   │   │   ├── ProductForm.tsx
│   │   │   ├── OrderTable.tsx
│   │   │   ├── ThemeCustomizer.tsx
│   │   │   ├── ComponentSelector.tsx
│   │   │   ├── DomainManager.tsx
│   │   │   ├── StripeConnectButton.tsx
│   │   │   └── (20 more...)
│   │   │
│   │   └── providers/
│   │       ├── ThemeProvider.tsx
│   │       ├── CartProvider.tsx
│   │       └── ToastProvider.tsx
│   │
│   ├── lib/
│   │   ├── api/                        # API client
│   │   │   ├── client.ts
│   │   │   ├── tenants.ts
│   │   │   ├── products.ts
│   │   │   ├── orders.ts
│   │   │   └── domains.ts
│   │   ├── hooks/                      # React hooks
│   │   │   ├── useTenant.ts
│   │   │   ├── useCart.ts
│   │   │   ├── useProducts.ts
│   │   │   └── (8 more...)
│   │   ├── utils/
│   │   │   ├── theme.ts
│   │   │   ├── format.ts
│   │   │   ├── validation.ts
│   │   │   └── cn.ts
│   │   └── constants.ts
│   │
│   ├── styles/
│   │   ├── globals.css
│   │   └── theme-base.css
│   │
│   └── types/
│       ├── tenant.ts
│       ├── product.ts
│       ├── order.ts
│       └── api.ts
```

```
│
├── tests/
│   ├── unit/
│   └── e2e/
│       └── playwright/
│
├── proxy.ts                                # Next.js 16 proxy (!)
├── next.config.ts
├── tailwind.config.ts
├── tsconfig.json
└── package.json
```

Frontend File Count: ~120 files

Implementation Timeline

Total Duration: 22 weeks (5.5 months)

Phase 1: Foundation (Weeks 1-4)

Week 1: Backend Setup

- [ ] Initialize Go project with Chi
- [ ] Set up Railway PostgreSQL (production + staging)
- [ ] Configure GitHub Actions CI/CD
- [ ] Implement base middleware (logging, CORS, request ID)
- [ ] Create health check endpoint

- **Deliverable:** API responds to /health

Week 2: Database Foundation

- [ ] Create all table schemas (14 migrations)
- [ ] Implement table partitioning (products, variants, orders)
- [ ] Add all indexes (covering indexes strategy)
- [ ] Set up RLS policies
- [ ] Seed 2 test tenants with products

- **Deliverable:** Database ready with test data

Week 3: Authentication & Multi-Tenancy

- [ ] Integrate Clerk SDK
- [ ] Implement Clerk webhook handlers (user sync)
- [ ] Build tenant resolution middleware
- [ ] Create tenant CRUD endpoints
- [ ] Test RBAC (admin, customer roles)
- **Deliverable:** Auth working, tenant isolation verified

Week 4: Core Product API

- [ ] Product CRUD endpoints
- [ ] Variant management
- [ ] Image upload to R2
- [ ] Integrate Meilisearch
- [ ] Product search endpoint (with Postgres FTS fallback)
- **Deliverable:** Product management API complete

Phase 2: E-Commerce Core (Weeks 5-8)

Week 5: Cart & Inventory

- [ ] Cart validation endpoint
- [ ] Inventory reservation logic
- [ ] Stock tracking (total vs reserved)
- [ ] Low stock alerts
- [ ] Inventory decrement on order
- **Deliverable:** Cart system functional

Week 6: Checkout & Payments

- [ ] Stripe Connect integration
- [ ] Checkout session creation
- [ ] Payment webhook handling (idempotency)
- [ ] Order creation on successful payment
- [ ] Stripe Tax integration
- **Deliverable:** End-to-end checkout works

Week 7: Order Management

- [ ] Order query endpoints
- [ ] Guest order access (tokens)
- [ ] Admin order dashboard API

- [ ] Shippo integration (shipping rates)
- [ ] Order fulfillment workflow
- **Deliverable:** Order management complete

Week 8: Email & Notifications

- [ ] AWS SES configuration
- [ ] Order confirmation emails
- [ ] Shipping notification emails
- [ ] Low stock admin alerts
- [ ] Email templates (HTML + text)
- **Deliverable:** Email system working

Phase 3: Frontend Development (Weeks 9-14)

Week 9: Next.js Setup

- [ ] Initialize Next.js 16 project
- [ ] Configure proxy.ts (multi-domain routing)
- [ ] Set up Vercel Edge Config
- [ ] Create tenant routing ([domain])
- [ ] Build API client layer
- **Deliverable:** Next.js routing works

Week 10: Base Components

- [ ] Button, Input, Card, Modal (15 components)
- [ ] Layout components
- [ ] Loading states & skeletons
- [ ] Error boundaries
- [ ] Toast notifications
- **Deliverable:** Component library ready

Week 11: Component Templates

- [ ] 5 Hero variants
- [ ] 3 Navigation variants
- [ ] 4 Product grid variants
- [ ] 3 Footer variants
- [ ] Template configuration system
- **Deliverable:** All templates implemented

Week 12: Storefront Pages

- [ ] Homepage (dynamic components)
- [ ] Product listing page
- [ ] Product detail page
- [ ] Cart page
- [ ] Checkout flow (3 steps)

- **Deliverable:** Complete storefront

Week 13: Admin Dashboard - Products

- [ ] Admin layout & navigation
- [ ] Product management table
- [ ] Product creation form
- [ ] Image uploader (5 images)
- [ ] Bulk actions

- **Deliverable:** Product admin complete

Week 14: Admin Dashboard - Settings

- [ ] Order management interface
- [ ] Order detail & fulfillment
- [ ] Theme customizer (live preview)
- [ ] Component selector & editor
- [ ] Domain management UI
- [ ] Stripe Connect onboarding
- [ ] Tax settings (Stripe Tax)

- **Deliverable:** Full admin dashboard

Phase 4: Testing & Optimization (Weeks 15-17)

Week 15: Performance Optimization

- [ ] Optimize Meilisearch indexes
- [ ] Implement query caching (Redis)
- [ ] Add covering indexes (missing ones)
- [ ] Optimize proxy.ts (Edge Config)
- [ ] Pre-generate theme CSS files

- **Deliverable:** <100ms P95 queries

Week 16: Testing

- [ ] Unit tests (Go services)
- [ ] Integration tests (API endpoints)
- [ ] E2E tests (Playwright - checkout flow)
- [ ] Load testing (k6 - 100 concurrent users)
- [ ] Security audit (SQL injection, XSS)
- **Deliverable:** Test coverage >80%

Week 17: Data Migration

- [ ] Write migration scripts (Supabase → Railway)
- [ ] Migrate images (Supabase Storage → R2)
- [ ] Migrate users (Supabase Auth → Clerk)
- [ ] Dual-write middleware (safety net)
- [ ] Data integrity validation
- **Deliverable:** Migration tooling ready

Phase 5: Launch Preparation (Weeks 18-22)

Week 18: Staging Deployment

- [ ] Deploy to Railway staging
- [ ] Deploy to Vercel staging
- [ ] Configure staging domains
- [ ] Run smoke tests
- [ ] Performance benchmarks
- **Deliverable:** Staging environment live

Week 19: Documentation

- [ ] API documentation (OpenAPI/Swagger)
- [ ] Admin user guide (screenshots)
- [ ] Deployment runbook
- [ ] Monitoring dashboards
- [ ] On-call procedures
- **Deliverable:** Complete documentation

Week 20: Gradual Rollout

- [ ] 10% traffic → new system
- [ ] Monitor errors & latencies
- [ ] 50% traffic → new system

- [ ] 100% traffic → new system
- [ ] Decommission old Supabase

- **Deliverable:** Full production launch

Week 21: First Tenant Onboarding

- [ ] Onboard first paying tenant
- [ ] Set up their domain
- [ ] Configure Stripe Connect
- [ ] Upload their products
- [ ] Test end-to-end order

- **Deliverable:** First customer live!

Week 22: Buffer & Polish

- [ ] Fix critical bugs
- [ ] Optimize slow queries
- [ ] Improve admin UX
- [ ] Add missing features
- [ ] Retrospective

- **Deliverable:** Stable production system

Development Workflow

Local Development

```
# Terminal 1: Start services
docker-compose up

# Terminal 2: Backend
cd backend
make dev

# Terminal 3: Frontend
cd frontend
npm run dev

# Terminal 4: Worker
cd backend
make worker

# Access
# Frontend: http://localhost:3000
# Backend:  http://localhost:8080
# Meilisearch: http://localhost:7700
```

Git Workflow

```
main         → Production
  ├─ staging → Staging environment
  └─ dev     → Development

Feature branches:
  ├─ feature/product-search
  ├─ feature/stripe-connect
  └─ bugfix/cart-quantity
```

PR Process

1. Create feature branch from `dev`
2. Implement feature + tests
3. Open PR → auto-preview environment (Vercel + Railway)

4. Code review (2 approvals required)
5. Merge to `dev`
6. Test in dev environment
7. Merge to `staging` (weekly)
8. Test in staging (2 days)
9. Merge to `main` (production deploy)

CI/CD Pipeline

```
# .github/workflows/backend-deploy.yml
on: [push]

jobs:
  test:
    - Run Go tests
    - Run linter (golangci-lint)
    - Check migrations

  deploy:
    if: branch == 'main' || branch == 'staging'
    - Build Docker image
    - Push to Railway
    - Run smoke tests
    - Notify Slack
```

Deployment Strategy

Production Infrastructure

Railway (Backend):

```
rdk-production/
  ├── api-service      (Go API)
  ├── meilisearch      (Search)
  └── postgres         (Database)
```

Vercel (Frontend):

```
rdk-frontend (Pro plan)
  ├── main branch      → Production domains
  └── staging branch   → staging.*.com
```

Environment Variables

Production (.env.production):

```
# Environment
ENV=production

# Database
DATABASE_URL=postgresql://...

# Redis
UPSTASH_REDIS_URL=...
UPSTASH_REDIS_TOKEN=...
REDIS_DB=0

# Clerk
CLERK_SECRET_KEY=sk_live_...
CLERK_PUBLISHABLE_KEY=pk_live_...

# Stripe
STRIPE_SECRET_KEY=sk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...

# Vercel API (domain management)
VERCEL_API_TOKEN=...
VERCEL_PROJECT_ID=...
VERCEL_TEAM_ID=...

# Meilisearch
MEILISEARCH_URL=https://meilisearch-prod.railway.app
MEILISEARCH_MASTER_KEY=...

# Storage
R2_ENDPOINT=https://....r2.cloudflarestorage.com
R2_ACCESS_KEY_ID=...
R2_SECRET_ACCESS_KEY=...
R2_BUCKET=rdk-production

# Email
SES_SMTP_HOST=email-smtp.us-east-1.amazonaws.com
SES_SMTP_PORT=587
SES_SMTP_USER=...
SES_SMTP_PASS=...
SES_FROM_EMAIL=noreply@rdk.com
```

```
# Monitoring
SENTRY_DSN=...
```

Deployment Commands

```
# Backend
railway up --environment production

# Frontend
vercel --prod

# Database migration
railway run --environment production make migrate-up

# Seed component templates
railway run --environment production make seed-components
```

Health Checks

```
# API
curl https://api.rdk.com/health

# Frontend
curl https://sneakerhaven.com

# Database
railway connect postgres --environment production
\l  # List databases
```

Security & Performance

Security Checklist

- [x] **SQL Injection:** Parameterized queries (sqlc)
- [x] **XSS:** HTML escaping on all user input
- [x] **CSRF:** SameSite cookies
- [x] **Auth:** Clerk JWT validation on all authenticated routes

- [x] **RBAC:** Role checks (admin, customer)
- [x] **Rate Limiting:** Upstash Redis, per-tenant limits
- [x] **Secrets:** Environment variables, never in code
- [x] **HTTPS:** Vercel & Railway auto-SSL
- [x] **Tenant Isolation:** Partitions + RLS policies
- [x] **Webhook Verification:** Stripe signature validation
- [x] **PII Protection:** No logging of sensitive data
- [x] **DDoS:** Cloudflare bot protection

Performance Targets

Metric	Target	Strategy
API Latency (P95)	<100ms	Partitioning, indexes, caching
Search Latency (P95)	<200ms	Meilisearch (Postgres FTS fallback)
Proxy Overhead (P95)	<5ms	Vercel Edge Config
Page Load (P95)	<500ms	SSR, image optimization, caching
Database Queries	<50ms P95	Covering indexes, partitions
Uptime	99.9%	Railway HA, Vercel global CDN

Monitoring

```
# Prometheus metrics
- api_request_duration_seconds
- api_request_total
- db_query_duration_seconds
- db_connections_total
- cache_hit_total
- cache_miss_total
- search_latency_seconds
```

Alerts

```
- API P95 latency >100ms for 5min
- Database connections >80% for 3min
- Cache hit rate <80% for 10min
- Error rate >1% for 5min
- Search engine down for 1min
```

Migration Plan

Phase 1: Preparation (Week 17)

1. **Export Supabase data**
   - Products → CSV
   - Orders → CSV
   - Users → CSV
   - Images → List URLs
2. **Create migration scripts**
   - Transform CSV to SQL
   - Map Supabase IDs to ULIDs
   - Handle foreign key relationships
3. **Test migration on staging**
   - Verify data integrity
   - Check foreign keys
   - Validate image URLs

Phase 2: Dual-Write (Weeks 18-19)

```
// Write to both old and new system
func (s *ProductService) Create(ctx context.Context, product *Product) error {
    // Write to new system
    err := s.newRepo.Create(ctx, product)
    if err != nil {
        return err
    }

    // Mirror to old system (best effort)
    go func() {
        s.supabase.CreateProduct(product)
    }()

    return nil
}
```

Phase 3: Gradual Traffic Shift (Week 20)

```
Day 1-2:   10% traffic → new system
Day 3-5:   25% traffic → new system
Day 6-8:   50% traffic → new system
Day 9-12:  75% traffic → new system
Day 13-14: 100% traffic → new system
```

Phase 4: Decommission (Week 21)

1. Stop dual-write
2. Final data sync
3. Archive Supabase database
4. Cancel Supabase subscription
5. Update DNS (if needed)

Success Metrics

Launch Criteria (Week 20)

- [ ] 100% uptime for 72 hours (staging)
- [ ] <500ms P95 page load
- [ ] All E2E tests passing
- [ ] Zero critical security vulnerabilities
- [ ] 1 pilot tenant live
- [ ] Successful test order (end-to-end)
- [ ] Admin dashboard functional
- [ ] Stripe Connect working

30-Day Goals (Month 2)

- [ ] 3-5 paying tenants
- [ ] 99.9% uptime
- [ ] <10 support tickets per week
- [ ] $500+ MRR
- [ ] <2% cart abandonment rate
- [ ] 95%+ customer satisfaction (CSAT)

6-Month Goals (Month 7)

- [ ] 20+ paying tenants
- [ ] 50K total MAU across tenants
- [ ] $3,000+ MRR
- [ ] <1% error rate
- [ ] 10+ component templates
- [ ] Self-service onboarding

12-Month Goals (Month 13)

- [ ] 50+ paying tenants
- [ ] 200K total MAU
- [ ] $10,000+ MRR
- [ ] 99.95% uptime
- [ ] API public beta

- [ ] Mobile app beta

SEO Implementation (v2)

SEO Strategy Overview

Every tenant storefront must be independently discoverable and rank well for their niche (brand-specific sneaker searches, local pickup, etc.). SEO is critical for organic traffic acquisition.

On-Page SEO

Meta Tags (Dynamic per Page)

```tsx
// app/[domain]/products/[slug]/page.tsx
export async function generateMetadata({ params }): Promise<Metadata> {
  const product = await getProduct(params.slug);
  const tenant = await getTenant(params.domain);

  return {
    title: `${product.title} - ${product.brand} | ${tenant.name}`,
    description: product.meta_description || `Shop authentic ${product.brand} ${product.model} a
    keywords: `${product.brand}, ${product.model}, ${product.colorway}, sneakers, ${tenant.name

    // Open Graph
    openGraph: {
      title: product.title,
      description: product.description,
      images: [{ url: product.images[0].url }],
      type: 'product',
      siteName: tenant.name,
    },

    // Twitter
    twitter: {
      card: 'summary_large_image',
      title: product.title,
      description: product.description,
      images: [product.images[0].url],
    },

    // Product Schema
    other: {
      'product:price:amount': (product.base_price_cents / 100).toString(),
      'product:price:currency': 'USD',
    },
  };
}
```

Structured Data (JSON-LD)

```tsx
// components/shared/ProductSchema.tsx
export function ProductSchema({ product, tenant }: Props) {
  const schema = {
    "@context": "https://schema.org",
    "@type": "Product",
    "name": product.title,
    "image": product.images.map(img => img.url),
    "description": product.description,
    "brand": {
      "@type": "Brand",
      "name": product.brand
    },
    "offers": {
      "@type": "AggregateOffer",
      "priceCurrency": "USD",
      "lowPrice": product.variants.reduce((min, v) => Math.min(min, v.price_cents), Infinity) /
      "highPrice": product.variants.reduce((max, v) => Math.max(max, v.price_cents), 0) / 100,
      "offerCount": product.variants.filter(v => v.quantity > 0).length,
      "availability": product.total_quantity > 0 ? "https://schema.org/InStock" : "https://scher
      "seller": {
        "@type": "Organization",
        "name": tenant.name
      }
    }
  };

  return (
    <script
      type="application/ld+json"
      dangerouslySetInnerHTML={{ __html: JSON.stringify(schema) }}
    />
  );
}
```

Organization Schema (Per Tenant)

```
// app/[domain]/layout.tsx
const organizationSchema = {
  "@context": "https://schema.org",
  "@type": "Store",
  "name": tenant.name,
  "url": `https://${tenant.slug}.rdk.com`,
  "logo": tenant.logo_url,
  "contactPoint": {
    "@type": "ContactPoint",
    "email": tenant.support_email,
    "contactType": "Customer Service"
  }
};
```

Technical SEO

Sitemap Generation (Per Tenant)

```
// app/[domain]/sitemap.ts
export default async function sitemap({ params }): Promise<MetadataRoute.Sitemap> {
  const tenant = await getTenant(params.domain);
  const products = await getProducts(tenant.id);

  return [
    {
      url: `https://${tenant.slug}.rdk.com`,
      lastModified: new Date(),
      changeFrequency: 'daily',
      priority: 1,
    },
    {
      url: `https://${tenant.slug}.rdk.com/products`,
      lastModified: new Date(),
      changeFrequency: 'daily',
      priority: 0.8,
    },
    ...products.map(product => ({
      url: `https://${tenant.slug}.rdk.com/products/${product.slug}`,
      lastModified: product.updated_at,
      changeFrequency: 'weekly' as const,
      priority: 0.6,
    })),
  ];
}
```

robots.txt (Per Tenant)

```
// app/[domain]/robots.ts
export default function robots(): MetadataRoute.Robots {
  return {
    rules: [
      {
        userAgent: '*',
        allow: '/',
        disallow: ['/admin/', '/account/', '/checkout/'],
      },
    ],
    sitemap: `https://${params.domain}/sitemap.xml`,
  };
}
```

Canonical URLs

```
// Prevent duplicate content across custom domains
<link rel="canonical" href={`https://${tenant.primary_domain}/products/${slug}`} />
```

Performance SEO

Core Web Vitals Optimization

- **LCP (Largest Contentful Paint):** <2.5s
  - Preload hero images
  - Use Next.js Image component with priority
  - CDN via Cloudflare
- **FID (First Input Delay):** <100ms
  - Defer non-critical JavaScript
  - Use React Server Components
- **CLS (Cumulative Layout Shift):** <0.1
  - Reserve space for images with aspect-ratio
  - Avoid layout shifts from ads/banners

Image Optimization

```
// All product images served via Cloudflare R2 + CDN
<Image
  src={product.images[0].url}
  alt={product.images[0].alt_text}
  width={600}
  height={600}
  priority={index === 0} // First image only
  sizes="(max-width: 768px) 100vw, 50vw"
/>
```

Content SEO (Admin Features)

Per-Product SEO Fields

```
ALTER TABLE products ADD COLUMN meta_title VARCHAR(60);
ALTER TABLE products ADD COLUMN meta_description VARCHAR(160);
ALTER TABLE products ADD COLUMN seo_keywords TEXT; -- Comma-separated
```

Admin SEO Editor

```
// Admin dashboard: Edit product SEO
<Form>
  <Input
    label="SEO Title (60 chars max)"
    name="meta_title"
    maxLength={60}
    placeholder="Nike Air Jordan 1 Bred - Size 10 | ShopName"
  />
  <Textarea
    label="Meta Description (160 chars max)"
    name="meta_description"
    maxLength={160}
    placeholder="Shop authentic Nike Air Jordan 1 'Bred' colorway..."
  />
  <Input
    label="Focus Keyword"
    name="focus_keyword"
    placeholder="jordan 1 bred"
  />
</Form>
```

SEO Score Preview (Admin)

- Title length check
- Description length check
- Keyword in title
- Keyword in description
- Image alt text present
- Internal linking suggestions

Local SEO (For Pickup/Local Stores)

LocalBusiness Schema

```
{
  "@context": "https://schema.org",
  "@type": "LocalBusiness",
  "name": "Sneaker Haven LA",
  "address": {
    "@type": "PostalAddress",
    "streetAddress": "123 Main St",
    "addressLocality": "Los Angeles",
    "addressRegion": "CA",
    "postalCode": "90001"
  },
  "geo": {
    "@type": "GeoCoordinates",
    "latitude": 34.0522,
    "longitude": -118.2437
  },
  "openingHours": "Mo-Sa 10:00-20:00"
}
```

Link Building Features

Auto-Generated Collection Pages

```
/products/nike-air-jordan
/products/yeezy
/products/new-arrivals
/products/under-200
```

Internal Linking

- Related products (same brand)
- You may also like (similar price)
- Recently viewed
- Breadcrumbs with schema markup

Analytics Integration

Google Search Console (Per Tenant)

- API endpoint: `POST /v1/admin/seo/submit-sitemap`
- Auto-submit sitemap to GSC on product publish
- Track impressions, clicks, CTR

SEO Performance Dashboard

```
// Admin dashboard widget
<SEOMetrics>
  <Metric label="Organic Traffic" value="2,341" change="+12%" />
  <Metric label="Avg Position" value="8.3" change="-1.2" />
  <Metric label="Indexed Pages" value="847" change="+23" />
  <Metric label="Core Web Vitals" value="Good" />
</SEOMetrics>
```

Implementation Timeline (Within v2)

Week 11 (Component Library):

- Add ProductSchema component
- Add OrganizationSchema to layout

Week 12 (Storefront Pages):

- Implement generateMetadata for all pages
- Add canonical URLs
- Optimize images with Next.js Image

Week 13 (Admin Dashboard - Part 1):

- Add SEO fields to product form
- SEO preview widget

Week 14 (Admin Dashboard - Part 2):

- Generate sitemap.xml dynamically
- robots.txt per tenant
- Google Search Console integration

Week 15 (Performance Optimization):

- Core Web Vitals optimization
- Lighthouse audit (target: 90+ score)
- Image lazy loading

Future Roadmap (v3 - Post-Launch)

Overview

v2 is the **core e-commerce platform**. v3 adds **content marketing, marketplace integrations, and visual builders** to help tenants grow beyond just product listings.

v3.1: Content & Blogging (Months 7-9)

Blog/Content System

```
CREATE TABLE posts (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL,

  title VARCHAR(255) NOT NULL,
  slug VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  excerpt TEXT,

  -- SEO
  meta_title VARCHAR(60),
  meta_description VARCHAR(160),

  -- Media
  featured_image_url TEXT,

  -- Organization
  category_id ULID,
  tags TEXT[], -- Array of tags

  -- Publishing
  status VARCHAR(20) DEFAULT 'draft',
  published_at TIMESTAMPTZ,
  author_id ULID REFERENCES tenant_users(id),

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),

  UNIQUE(tenant_id, slug)
);

CREATE TABLE post_categories (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL,
  name VARCHAR(100) NOT NULL,
  slug VARCHAR(100) NOT NULL,
```

```
  UNIQUE(tenant_id, slug)
);
```

Blog Features

- Rich text editor (TipTap or Lexical)
- Image upload & gallery
- SEO optimization per post
- Related posts
- Social sharing buttons
- Comments system (optional)
- RSS feed generation
- Blog sitemap

Admin Interface

```
/admin/blog/
  ├── posts/           # All posts (draft, published, scheduled)
  ├── new/             # Create new post
  ├── categories/      # Manage categories
  └── settings/        # Blog settings (RSS, comments, etc.)
```

SEO Benefits

- Fresh content for Google
- Long-tail keyword targeting
- Internal linking to products
- Build topical authority

Example Posts:

- "How to Style Air Jordan 1s: 5 Outfit Ideas"
- "Sneaker Care Guide: Keeping Your Kicks Fresh"
- "Nike vs Adidas: Which Brand is Right for You?"

v3.2: Landing Page Builder (Months 10-12)

Visual Page Builder (Wix-like)

- Drag-and-drop interface
- Pre-built sections (hero, features, testimonials, CTA)

- Custom landing pages for campaigns
- A/B testing support

Component Library Expansion

```
// New draggable components
- ImageWithText (side-by-side layout)
- TestimonialCarousel
- BeforeAfter (product comparisons)
- VideoEmbed (YouTube, Vimeo)
- Countdown Timer (limited drops)
- EmailCapture (newsletter signup)
- FAQ Accordion
- PricingTable (membership tiers)
```

Use Cases

- Product launch pages
- Seasonal sale campaigns
- Brand story pages
- Limited edition drops
- Referral program landing pages

Technical Implementation

```sql
CREATE TABLE landing_pages (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL,

  title VARCHAR(255) NOT NULL,
  slug VARCHAR(255) NOT NULL,

  -- Visual builder JSON
  page_config JSONB NOT NULL,

  -- SEO
  meta_title VARCHAR(60),
  meta_description VARCHAR(160),

  -- Analytics
  view_count INTEGER DEFAULT 0,
  conversion_count INTEGER DEFAULT 0,

  status VARCHAR(20) DEFAULT 'draft',
  published_at TIMESTAMPTZ,

  UNIQUE(tenant_id, slug)
);
```

Page Config Structure:

```
{
  "sections": [
    {
      "id": "hero-1",
      "type": "hero_video",
      "config": {
        "videoUrl": "...",
        "heading": "Limited Edition Drop",
        "ctaText": "Shop Now",
        "ctaLink": "/products/jordan-1-travis-scott"
      }
    },
    {
      "id": "features-1",
      "type": "features_grid",
      "config": {
        "items": [
          { "icon": "truck", "title": "Free Shipping", "description": "..." },
          { "icon": "shield", "title": "Authenticity Guaranteed", "description": "..." }
        ]
      }
    }
  ]
}
```

v3.3: Marketplace Integrations (Months 13-15)

Multi-Channel Selling

Enable tenants to list products on external marketplaces while managing inventory from one place.

Supported Platforms:

- eBay
- Poshmark
- Mercari
- Grailed
- StockX (price comparison only)

eBay Integration

```
CREATE TABLE marketplace_listings (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL,
  product_id ULID NOT NULL,
  variant_id ULID,

  marketplace VARCHAR(50) NOT NULL, -- ebay, poshmark, etc.
  external_listing_id VARCHAR(255), -- eBay listing ID

  -- Pricing
  marketplace_price_cents INTEGER NOT NULL,
  marketplace_shipping_cents INTEGER,

  -- Status
  status VARCHAR(50) DEFAULT 'pending', -- pending, active, sold, ended
  listed_at TIMESTAMPTZ,
  ended_at TIMESTAMPTZ,

  -- Sync
  last_synced_at TIMESTAMPTZ,
  sync_error TEXT
);
```

Inventory Synchronization

- Sell on eBay → Auto-decrement RDK inventory
- Sell on RDK → Auto-end eBay listing
- Prevent double-selling

Admin Interface

```
/admin/marketplaces/
  ├── connect/          # Connect eBay, Poshmark accounts
  ├── listings/         # View all marketplace listings
  ├── sync/             # Manual sync trigger
  └── settings/         # Default pricing rules
```

Pricing Rules

```
// Admin can set markup rules
{
  "ebay": {
    "markup_percentage": 10,      // List 10% higher than RDK price
    "shipping_cost_cents": 1000   // $10 shipping on eBay
  },
  "poshmark": {
    "markup_percentage": 15,
    "free_shipping": true
  }
}
```

v3.4: Email Marketing (Months 16-18)

Built-in Email Campaigns

- Welcome series (new subscribers)
- Abandoned cart recovery
- Product restock alerts
- New arrival announcements
- Birthday discounts

Email Builder

- Drag-and-drop email editor
- Product insertion (pull from catalog)
- Discount code generation
- A/B testing

Integration with Klaviyo/Mailchimp

- API sync subscribers
- Trigger campaigns on events (order placed, cart abandoned)
- Track revenue attribution

```sql
CREATE TABLE email_campaigns (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL,

  name VARCHAR(255) NOT NULL,
  subject VARCHAR(255) NOT NULL,
  preview_text VARCHAR(255),

  -- Content
  html_content TEXT NOT NULL,

  -- Audience
  segment VARCHAR(50), -- all_subscribers, recent_customers, cart_abandoners

  -- Scheduling
  status VARCHAR(50) DEFAULT 'draft',
  scheduled_at TIMESTAMPTZ,
  sent_at TIMESTAMPTZ,

  -- Analytics
  recipients_count INTEGER DEFAULT 0,
  opens_count INTEGER DEFAULT 0,
  clicks_count INTEGER DEFAULT 0,
  conversions_count INTEGER DEFAULT 0,
  revenue_cents INTEGER DEFAULT 0
);
```

v3.5: Customer Loyalty & Rewards (Months 19-21)

Points System

```
CREATE TABLE loyalty_points (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL,
  user_id ULID NOT NULL,

  points_balance INTEGER DEFAULT 0,
  lifetime_points_earned INTEGER DEFAULT 0,

  tier VARCHAR(50) DEFAULT 'bronze', -- bronze, silver, gold, platinum
  tier_updated_at TIMESTAMPTZ
);

CREATE TABLE loyalty_transactions (
  id ULID PRIMARY KEY,
  tenant_id ULID NOT NULL,
  user_id ULID NOT NULL,

  points_change INTEGER NOT NULL, -- +100 or -50
  reason VARCHAR(50) NOT NULL, -- purchase, redemption, referral

  order_id ULID,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Reward Rules (Admin Configurable)

- Earn 1 point per $1 spent
- 100 points = $5 discount
- Birthday bonus: 500 points
- Referral bonus: 250 points per referred customer

Tier Benefits

- **Bronze:** Standard shipping
- **Silver:** Free shipping on orders $50+
- **Gold:** Free shipping + early access to drops
- **Platinum:** Free shipping + early access + exclusive discounts

v3.6: Mobile App (Months 22-24)

React Native App (Per Tenant)

- White-label mobile app
- Push notifications for new drops
- Mobile-optimized checkout
- Barcode scanner (for in-store pickup)

App Store Listing:

- Each tenant can publish their own branded app
- Or use RDK's multi-tenant app with tenant switcher

Features

- Wishlist sync
- Saved payment methods
- Order tracking
- Loyalty points display
- Push notifications (low stock alerts, flash sales)

Risk Mitigation

Technical Risks

Risk	Likelihood	Impact	Mitigation
Clerk pricing increase	Medium	High	Build fallback auth, negotiate contract
Railway performance	Low	High	Monitor closely, DigitalOcean backup plan
Meilisearch scaling	Medium	Medium	Postgres FTS fallback always available
Database conn exhaustion	Medium	High	Connection pooling, read replicas
Stripe API changes	Low	Medium	Pin API version, monitor changelog

Business Risks

Risk	Likelihood	Impact	Mitigation
Slow tenant acquisition	Medium	High	Pre-launch waitlist, referral incentives
High tenant churn	Medium	High	Onboarding support, success tracking
Feature requests exceed capacity	High	Medium	Public roadmap, voting system
Competitor launches similar	Medium	Medium	Focus on UX & customization depth

Operational Risks

Risk	Likelihood	Impact	Mitigation
Solo developer burnout	Medium	High	Automate testing, clear docs, consider hiring
Data loss incident	Low	Critical	Automated backups, quarterly DR drills
Security breach	Low	Critical	Penetration testing, bug bounty, incident response plan

Conclusion

This plan provides a comprehensive roadmap for rebuilding RDK into a scalable multi-tenant platform.

Key Strengths:

- ✅ Proven technology stack (Go, PostgreSQL, Next.js 16)
- ✅ Cost-effective infrastructure ($87 production + 5$ staging)
- ✅ Strong tenant isolation (partitioning + RLS)
- ✅ 10-50x faster search (Meilisearch)
- ✅ <1ms domain routing (Edge Config)
- ✅ Comprehensive folder structure (200+ files documented)

- ✅ Realistic 22-week timeline
- ✅ Multiple revenue tiers ($29—199/month)

Critical Success Factors:

1. Follow the timeline strictly (no feature creep)
2. Implement database partitioning from Day 1
3. Test tenant isolation thoroughly
4. Monitor query performance continuously
5. Keep staging costs minimal ($5/month)
6. Document everything as you build

Next Steps:

1. Review this plan thoroughly
2. Set up development environment (Week 1)
3. Begin backend foundation (Week 1-2)
4. Schedule weekly progress reviews
5. Commit to 22-week timeline

Ready to build! 🚀

Document Version: 3.0 (Final)
Total Pages: 60+
Total Words: 25,000+
Status: Locked & Ready for Implementation
Last Updated: February 9, 2026